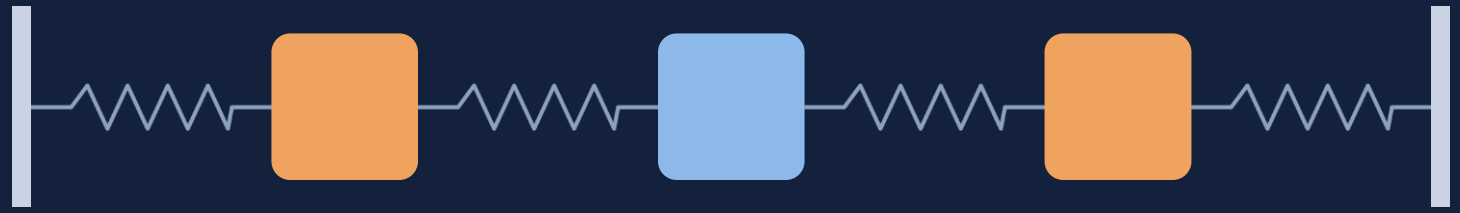
Osciladores armónicos acoplados

De la ecuación $M\ddot{x} = -Kx$ a una librería publicada en PyPI, con programación orientada a objetos, el patrón Strategy y una interfaz en tkinter.



Carlos Andrés Gómez Vasco · Desarrollo de software

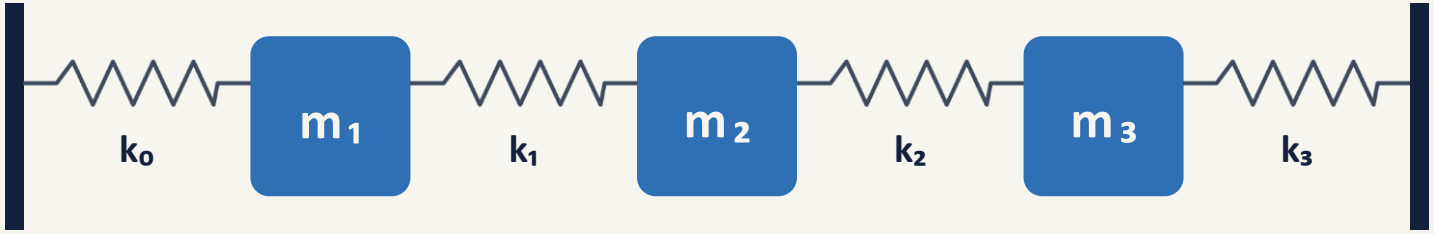
AGENDA

Siete etapas en 120 minutos

Inicio · duración	Etapa	Entregable
0:00 · 15 min	1. Física: modelo y modos normales	Ecuación $M\ddot{x} = -Kx$
0:15 · 15 min	2. Diseño: capas y patrón Strategy	Diagrama de clases
0:30 · 20 min	3. Modelo: SistemaOsciladores	Matrices, energía y modos
0:50 · 20 min	4. Estrategias de solución	Cuatro métodos y una fábrica
1:10 · 15 min	5. Interfaz gráfica en tkinter	GUI.py
1:25 · 10 min	6. Pruebas con pytest	tests/test_core.py
1:35 · 20 min	7. Empaquetar y publicar en PyPI	pip install osciladores-acoplados
1:55 · 5 min	Cierre	Retos para la próxima clase

ETAPA 1 · FÍSICA · 15 MIN

Una cadena de N masas y N + 1 resortes



Newton para la masa i

$$m_i \ddot{x}_i = -k_{i-1}(x_i - x_{i-1}) - k_i (x_i - x_{i+1})$$

$x_0 = x_{n+1} = 0$ son las paredes. Un resorte con $k = 0$ es un extremo libre.

Las N ecuaciones juntas

$$M \ddot{x} = -K x$$

Un sistema lineal de segundo orden: todo el problema cabe en dos matrices.

ETAPA 1 · FÍSICA · 15 MIN

Las matrices M y K para N = 3

$$M = \begin{pmatrix} m_1 & 0 & 0 \\ 0 & m_2 & 0 \\ 0 & 0 & m_3 \end{pmatrix} \qquad K = \begin{pmatrix} k_0+k_1 & -k_1 & 0 \\ -k_1 & k_1+k_2 & -k_2 \\ 0 & -k_2 & k_2+k_3 \end{pmatrix}$$

Modos normales

Proponer $x(t) = v\cos(\omega t)$ convierte el problema en $K\,v = \omega^2 M\,v$: un problema de autovalores generalizado.

Truco numérico

$A = M^{-½}\,K\,M^{-½}$ es simétrica: numpy.linalg.eigh(A) da ω^2 y u, y los modos son $v = M^{-½}\,u$.

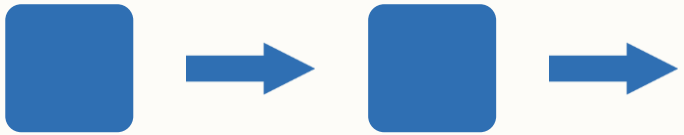
Diagonal (naranja): resortes que tocan la masa · fuera de la diagonal (azul): acoples entre vecinas

ETAPA 1 · FÍSICA · 15 MIN

Un resultado de libro para validar el código

Dos masas iguales m y tres resortes iguales k, extremos fijos:

Modo 1 • en fase



$$\omega_1 = \sqrt{(k/m)}$$

Las masas se mueven juntas; el resorte central no se estira.

Modo 2 • en contrafase



$$\omega_2 = \sqrt{(3k/m)}$$

Se mueven en sentidos opuestos; el resorte central trabaja doble.

Este caso es la prueba test_frecuencias_dos_masas_tres_resortes de la etapa 6

ETAPA 2 · DISEÑO · 15 MIN

Separar la lógica de la interfaz



core.py nunca importa tkinter. Se puede probar y reutilizar sin abrir ventanas.

La GUI no hace física. Lee texto, llama a la librería y dibuja el resultado.

Las flechas bajan. Una capa sólo conoce a la de abajo.

ETAPA 2 · DISEÑO · 15 MIN

Una tarea, cuatro algoritmos

SIN PATRÓN

```
def resolver(sistema, x0, v0, metodo):
    if metodo == "euler":
        ...
    elif metodo == "verlet":
        ...
    elif metodo == "rk4":
        ...
    elif metodo == "exacto":
        ...
    # método nuevo = editar esta función
```

Todos los algoritmos en una función que crece sin fin, y la GUI debe conocer cada nombre.

CON STRATEGY

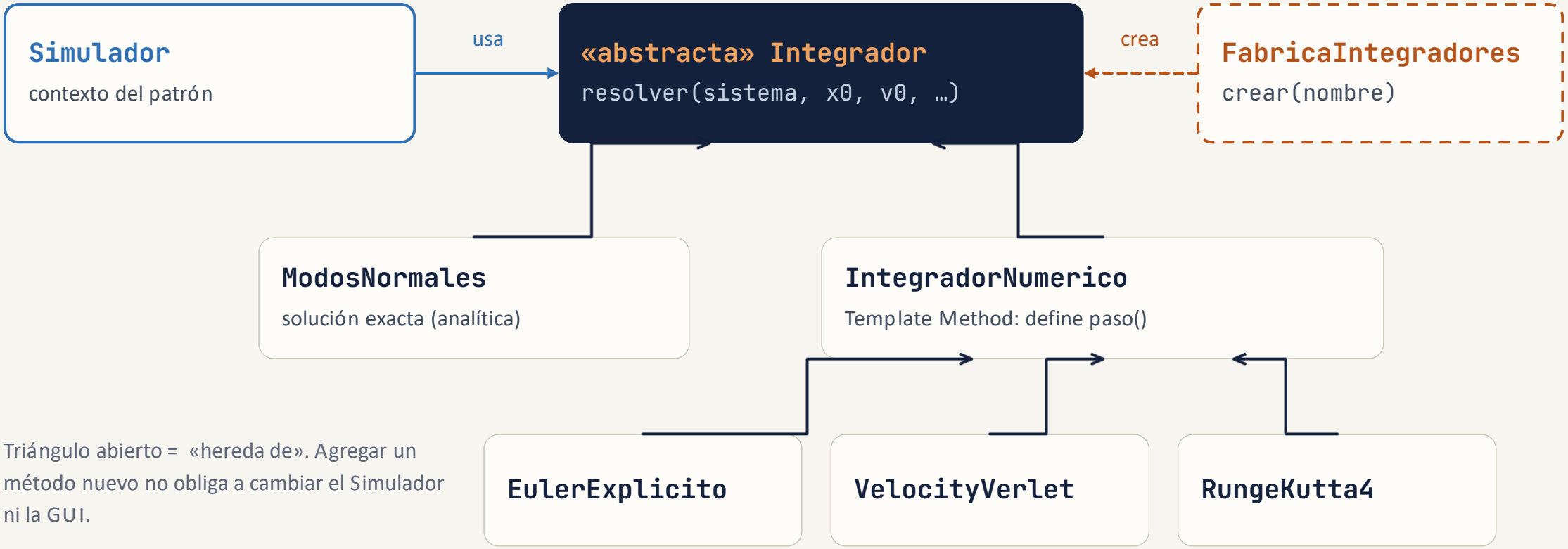
```
class VelocityVerlet(IntegradorNumerico):
    nombre = "Velocity Verlet"
    def paso(self, sistema, x, v, dt): ...

sim = Simulador(sistema, VelocityVerlet())
res1 = sim.ejecutar(x0, v0, 20, 0.01)
sim.estrategia = RungeKutta4()
res2 = sim.ejecutar(x0, v0, 20, 0.01)
# método nuevo = clase nueva
```

Cada método vive aislado y se cambia en una línea: el resto del código no se entera.

ETAPA 2 · DISEÑO · 15 MIN

Patrón Strategy: métodos intercambiables



Triángulo abierto = «hereda de». Agregar un método nuevo no obliga a cambiar el Simulador ni la GUI.

ETAPA 2 · DISEÑO · 15 MIN

¿Por qué Strategy y no otra cosa?

Comparar es la meta

Mismo sistema, otro método y la misma salida: un Resultado.

GUI desacoplada

El menú se llena solo con la lista de la fábrica.

Física ≠ numérica

Un sistema de tres masas no es «un sistema RK4».

Pruebas y retos

Cada método se valida contra el exacto; el amortiguamiento lo extiende.

ALTERNATIVAS DESCARTADAS

State

Mismo esqueleto, otra intención: aquí no hay transiciones; elige el usuario.

Sólo Template Method

Sirve para los numéricos, pero la solución exacta no avanza paso a paso.

Sólo funciones

Más corto en Python, pero sin nombre propio ni utilidades compartidas.

ETAPA 2 · DISEÑO · 15 MIN

Un patrón principal y tres de apoyo

Patrón	Dónde está	Qué resuelve
Strategy	Integrador + Simulador	Cambiar de método sin tocar lo demás
Template Method	IntegradorNumerico → paso()	El bucle temporal se escribe una vez
Fábrica con registro	FabricaIntegradores	Crear métodos por nombre desde la GUI
Observador	Simulador.suscribir()	La GUI reacciona sin acoplarse

PRINCIPIO ABIERTO/CERRADO

Agregar un integrador nuevo = escribir una clase y decorarla con @FabricaIntegradores.registrar. Ningún archivo existente cambia.

ETAPA 3 · MODELO · 20 MIN

Antes del código: el Python que vamos a leer

CÓDIGO	QUÉ SIGNIFICA
`class Nombre:`	Un molde, como el plano de una casa. Nombre(…) construye un objeto concreto: una casa hecha con ese plano.
`self`	Dentro de la clase significa «yo mismo»: el objeto concreto que está ejecutando el método.
`def __init__(self, ...):`	El constructor. Se ejecuta solo al crear el objeto y guarda sus datos iniciales.
`self._m`	Un atributo: un dato que el objeto recuerda. El guion bajo avisa «es interno, no lo cambies desde afuera».
`@decorador`	Una etiqueta encima de una función que cambia cómo se comporta (por ejemplo @property o @staticmethod).
`np.array([1, 2, 3])`	Un arreglo de NumPy: una lista de números con la que se hacen cuentas de una vez, sin ciclos.

ETAPA 3 · MODELO · 20 MIN

La clase SistemaOsciladores

```python
class SistemaOsciladores:
    def __init__(self, masas, constantes):
        self._m = np.asarray(masas, dtype=float)
        self._k = np.asarray(constantes, dtype=float)
        if self._k.size != self._m.size + 1:
            raise ValueError("Se necesitan N+1 constantes")
        self._K = self._construir_rigidez(self._k)
    @staticmethod
    def _construir_rigidez(k):
        n = k.size - 1
        K = np.zeros((n, n))
        for i in range(n):
            K[i, i] = k[i] + k[i + 1]
            if i < n - 1:
                K[i, i + 1] = K[i + 1, i] = -k[i + 1]
        return K
```

Encapsular

Atributos _privados y propiedades que devuelven copias.

Validar al construir

Masas > 0, k ≥ 0 y N + 1 resortes: nunca nace inválido.

Constructor alternativo

uniforme(n, masa, k) crea una cadena de elementos iguales.

ETAPA 3 · MODELO · 20 MIN

SistemaOsciladores, paso a paso

CÓDIGO	QUÉ SIGNIFICA
`np.asarray(masas, dtype=float)`	Convierte la lista del usuario, como [1, 1, 1], en un arreglo de decimales.
`if … : raise ValueError(…)`	Si los datos no tienen sentido, se detiene con un mensaje claro.
`@staticmethod`	Función de la clase que no usa self: sólo trabaja con lo que recibe.
`for i in range(n):`	Repite el bloque con i = 0, 1, …, n − 1: una vez por cada fila de K.
`K[i, i + 1] = K[i + 1, i] = -k[i + 1]`	Escribe el mismo valor en dos casillas: por eso K queda simétrica.

EJEMPLO A MANO: 2 MASAS

```
k = [10, 5, 10]   → n = 2
i = 0: K[0,0] = 10 + 5 = 15
       K[0,1] = K[1,0] = -5
i = 1: K[1,1] = 5 + 10 = 15

K = [[ 15, -5],
     [ -5, 15]]
```

ETAPA 3 · MODELO · 20 MIN

Modos normales con siete líneas de NumPy

```
def modos_normales(self):
    m_inv_raiz = 1.0 / np.sqrt(self._m)                      # M^(-1/2)
    A = (m_inv_raiz[:, None] * self._K) * m_inv_raiz[None, :]  # simétrica
    w2, U = np.linalg.eigh(A)
    w2 = np.clip(w2, 0.0, None)          # elimina -1e-16 numéricos
    phi = m_inv_raiz[:, None] * U        # modos con Φᵀ M Φ = I
    return np.sqrt(w2), phi
```

Frecuencias

frecuencias_hz() devuelve f = ω / 2π para cada modo.

Excitar un modo

condicion_modo(j) da un x(0) que activa sólo el modo j.

Comprobar

Con dos masas iguales deben salir √(k/m) y √(3k/m).

ETAPA 3 · MODELO · 20 MIN

Las siete líneas, traducidas

CÓDIGO	QUÉ SIGNIFICA
`1.0 / np.sqrt(self._m)`	Calcula $1/\sqrt{m_i}$ para todas las masas a la vez, sin escribir un ciclo.
`v[:, None]` y `v[None, :]`	Ponen el vector de pie (columna) o acostado (fila). Al multiplicar, cada fila y columna de K se escala: eso es $M^{-½}\ K\ M^{-½}$.
`np.linalg.eigh(A)`	Resuelve autovalores de una matriz simétrica: da los ω^2, de menor a mayor, y los vectores propios.
`np.clip(w2, 0.0, None)`	Cambia por 0 los negativos diminutos (−1e−16) que deja el redondeo, para que la raíz no falle.
`return np.sqrt(w2), phi`	Devuelve dos cosas: las frecuencias ω y la matriz de modos, un modo por columna.

COMPROBACIÓN EN CONSOLA

```
>>> s = SistemaOsciladores(
...     [1, 1], [10, 10, 10])
>>> w, phi = s.modos_normales()
>>> w
array([3.162, 5.477])
>>> np.sqrt([10, 30])
array([3.162, 5.477])
```

Coinciden con √(k/m) y √(3k/m) del tablero.

ETAPA 3 · MODELO · 20 MIN

Un objeto para el resultado

```python
@dataclass(frozen=True)
class Resultado:
    t: np.ndarray          # (T,)
    x: np.ndarray          # (T, N)
    v: np.ndarray          # (T, N)
    energia: np.ndarray    # (T,)
    metodo: str

    @property
    def error_energia_relativo(self):
        return (self.energia - self.energia[0]) / self.energia[0]

    def a_csv(self, ruta): ...
```

Energía mecánica

$$E = \tfrac{1}{2}\, v^{T} M\, v + \tfrac{1}{2}\, x^{T} K\, x$$

Debe ser constante: es nuestro termómetro de calidad numérica.

frozen=True

El resultado es inmutable: la GUI puede graficarlo y guardarlo sin riesgo de alterarlo.

ETAPA 3 · MODELO · 20 MIN

Un Resultado es una tabla de números

RES.T Y RES.X PARA 3 MASAS, DT = 0.01 S

fila	t (s)	x_1 (m)	x_2 (m)	x_3 (m)
0	0.00	0.500	0.000	0.000
50	0.50	−0.232	0.235	0.076
100	1.00	−0.040	−0.293	0.079
2000	20.00	…	…	…

res.x.shape es (2001, 3): 2001 instantes, 3 masas. res.x[:, 0] es toda la columna de la masa 1.

CÓDIGO

```
@dataclass
```

```
frozen=True
```

```
@property
```

QUÉ SIGNIFICA

Escribe el constructor por nosotros: basta con listar los campos.

Congela el objeto: una vez creado, sus datos no se pueden cambiar.

Permite usar res.error_energia_relativo sin paréntesis, como si fuera un dato.

ETAPA 4 · ESTRATEGIAS · 20 MIN

La interfaz común y la estrategia exacta

```python
class Integrador(ABC):
    nombre = "abstracto"
    @abstractmethod
    def resolver(self, sistema, x0, v0, t_final, dt):
        """Devuelve un Resultado entre 0 y t_final."""

class ModosNormales(Integrador):
    nombre = "Modos normales (exacto)"
    def resolver(self, sistema, x0, v0, t_final, dt):
        t = self._malla_temporal(t_final, dt)
        w, phi = sistema.modos_normales()
        M = sistema.matriz_masa
        q0, p0 = phi.T @ M @ x0, phi.T @ M @ v0
        q = q0*np.cos(np.outer(t, w)) + p0/w*np.sin(np.outer(t, w))
        x = q @ phi.T                       # vuelve a coordenadas x
        return Resultado(t, x, v, sistema.energia(x, v), self.nombre)
```

Solución exacta

$$x(t) = \Sigma_j \phi_j \left[q_j(0) \cos \omega_j t + p_j(0)/\omega_j \sin \omega_j t \right]$$

con $q(0) = \Phi^T M x_0$ y $p(0) = \Phi^T M v_0$

Versión resumida

El archivo real también calcula v(t) y trata el caso $\omega = 0$ de la traslación rígida.

ETAPA 4 · ESTRATEGIAS · 20 MIN

La solución exacta en tres pasos

1 • Descomponer

`q0 = phi.T @ M @ x0`

¿Cuánto de cada modo hay al inicio? Como separar un acorde en sus notas.

2 • Evolucionar

`q0*cos(ωt) + p0/ω*sin(ωt)`

Cada modo es un oscilador simple e independiente, de solución conocida.

3 • Recombinar

`x = q @ phi.T`

Sumar los modos de nuevo para obtener la posición de cada masa.

CÓDIGO	QUÉ SIGNIFICA
`class Integrador(ABC):`	Clase abstracta: un contrato. Python no deja crear un Integrador «genérico», sólo clases que implementen resolver.
`A @ B`	Producto de matrices, como en álgebra lineal. No confundir con el @ de los decoradores.
`np.outer(t, w)`	Tabla con todos los productos $\omega_j \cdot t$: una fila por instante, una columna por modo.

ETAPA 4 · ESTRATEGIAS · 20 MIN

Los métodos numéricos sólo definen paso()

```python
class IntegradorNumerico(Integrador):          # Template Method
    def resolver(self, sistema, x0, v0, t_final, dt):
        ...                                     # reserva t, X, V
        for i in range(t.size - 1):
            X[i+1], V[i+1] = self.paso(sistema, X[i], V[i], dt)
        return Resultado(t, X, V, sistema.energia(X, V), self.nombre)

class VelocityVerlet(IntegradorNumerico):
    nombre = "Velocity Verlet"
    def paso(self, sistema, x, v, dt):
        a = sistema.aceleracion(x)
        x_n = x + v*dt + 0.5*a*dt**2
        a_n = sistema.aceleracion(x_n)
        return x_n, v + 0.5*(a + a_n)*dt
```

Euler explícito

x ← x + v dt y v ← v + a dt. Orden 1.

Velocity Verlet

Simplético, orden 2: la energía oscila sin derivar.

Runge-Kutta 4

Cuatro evaluaciones por paso, orden 4.

ETAPA 4 · ESTRATEGIAS · 20 MIN

Velocity Verlet como receta de física

CÓDIGO	QUÉ SIGNIFICA
`a = sistema.aceleracion(x)`	Paso 1. Con las posiciones actuales, calcular fuerzas y aceleraciones: $a = -M^{-1}K\,x$ (segunda ley de Newton).
`x_n = x + v*dt + 0.5*a*dt**2`	Paso 2. Mover las masas con la cinemática de siempre, $x = x_0 + v\,t + \frac{1}{2}\,a\,t^2$, durante un tiempo dt.
`a_n = sistema.aceleracion(x_n)`	Paso 3. Recalcular la aceleración en las posiciones nuevas.
`v + 0.5*(a + a_n)*dt`	Paso 4. Actualizar la velocidad con el promedio de la aceleración vieja y la nueva. Euler sólo usa la vieja: por eso falla.
`for i in range(t.size - 1):`	La clase base repite la receta 2000 veces; X[i+1] es la siguiente fila de la tabla de resultados.
`dt**2`	En Python ** es potencia: dt**2 significa dt^2. El asterisco simple * es la multiplicación.

ETAPA 4 · ESTRATEGIAS · 20 MIN

Mismo sistema, cuatro estrategias

Máximo de $|E - E_0| / E_0$ con 3 masas de 1 kg, k = 10 N/m, $x_1(0)$ = 0.5 m, 20 s y dt = 0.01 s:

8.9×10^{-16}

Modos normales

Precisión de máquina: es la referencia.

4.2×10^{2}

Euler explícito

La energía crece unas 400 veces: inestable.

6.2×10^{-4}

Velocity Verlet

Error acotado que oscila, sin deriva.

5.8×10^{-7}

Runge-Kutta 4

Muy preciso; deriva lenta en tiempos largos.

ETAPA 4 · ESTRATEGIAS · 20 MIN

La fábrica y el contexto

```
class FabricaIntegradores:
    _registro = {}
    @classmethod
    def registrar(cls, clase):
        cls._registro[clase.nombre] = clase
        return clase
    @classmethod
    def crear(cls, nombre):
        return cls._registro[nombre]()

# Uso: la estrategia se cambia en caliente
sim = Simulador(sistema, VelocityVerlet())
res = sim.ejecutar(x0=[0.5, 0, 0], v0=[0, 0, 0], t_final=20, dt=0.01)
sim.estrategia = FabricaIntegradores.crear("Runge-Kutta 4")
```

RETO DE 5 MINUTOS

Escriban EulerCromer: primero v ← v + a dt y luego x ← x + v dt con la v nueva.

Decórenla con

`@FabricaIntegradores.registrar`

y aparecerá sola en la lista de la GUI.

ETAPA 4 · ESTRATEGIAS · 20 MIN

La fábrica es una agenda de contactos

CÓDIGO	QUÉ SIGNIFICA
`_registro = {}`	Un diccionario vacío: guarda pares «nombre → clase», como una agenda guarda «nombre → teléfono».
`@classmethod … cls`	Método de la clase, no de un objeto: cls es la clase misma. Así existe un único registro compartido.
`cls._registro[clase.nombre] = clase`	Anota en la agenda: con la llave "Velocity Verlet" se guarda la clase VelocityVerlet.
`return clase`	Devolver la clase permite usar registrar como decorador encima de una clase nueva.
`cls._registro[nombre]()`	Busca la clase por su nombre; los paréntesis () crean un objeto nuevo de esa clase.
`sim.estrategia = …`	Cambia el método «en caliente»: mismo simulador, mismo sistema, otro algoritmo.

ETAPA 5 · INTERFAZ GRÁFICA · 15 MIN

Una ventana, un panel y cuatro pestañas



Entradas como texto

«1, 1, 1» pasa por leer_lista() y se vuelve un arreglo.

Errores con mensaje

La lógica lanza ValueError y la GUI lo muestra en un messagebox.

Gráficas y animación

matplotlib con FigureCanvasTkAgg; la animación usa tk.Canvas y after().

ETAPA 5 · INTERFAZ GRÁFICA · 15 MIN

La GUI sólo traduce y dibuja

```
def simular(self):
    try:
        self.sistema = self._leer_sistema()
        x0, v0, t_final, dt = self._leer_condiciones()
        estrategia = FabricaIntegradores.crear(self.var_metodo.get())
        self.simulador = Simulador(self.sistema, estrategia)
        self.simulador.suscribir(self._al_recibir_resultado)
        self.simulador.ejecutar(x0, v0, t_final, dt)
    except ValueError as error:
        messagebox.showerror("Datos inválidos", str(error))

def _al_recibir_resultado(self, resultado):     # observador
    self.resultado = resultado
    self._graficar_posiciones()
    self._graficar_energia([resultado])
```



ETAPA 5 · INTERFAZ GRÁFICA · 15 MIN

La GUI, en lenguaje cotidiano

CÓDIGO	QUÉ SIGNIFICA
`try: … except ValueError as error:`	«Intenta esto; si los datos fallan, no cierres el programa: haz esto otro». El problema queda guardado en error.
`self.var_metodo.get()`	Lee lo que el usuario eligió. Las variables de tkinter se leen con .get() y se escriben con .set().
`suscribir(self._al_recibir_resultado)`	Sin paréntesis: se entrega la función sin ejecutarla. Es dejar el teléfono para que la lógica llame al terminar (callback).
`messagebox.showerror(…)`	Abre una ventanita con el mensaje que escribió la lógica, por ejemplo «Con 3 masas se necesitan 4 constantes».
`self._graficar_posiciones()`	Dibuja con matplotlib dentro de la ventana; la GUI sólo dibuja lo que la lógica ya calculó.

ETAPA 6 · PRUEBAS · 10 MIN

Las pruebas comprueban la física

```python
def test_frecuencias_dos_masas_tres_resortes():
    m, k = 2.0, 8.0
    sis = SistemaOsciladores([m, m], [k, k, k])
    w, _ = sis.modos_normales()
    assert np.allclose(w, [np.sqrt(k/m), np.sqrt(3*k/m)])

def test_verlet_conserva_energia_y_euler_no():
    ...
    assert max(abs(ver.error_energia_relativo)) < 1e-2
    assert eul.error_energia_relativo[-1] > 0.5
```

```
$ pytest -v
================ 14 passed ================
```

Qué probar en física

- Casos con solución analítica
- Leyes de conservación
- Propiedades: $\Phi^T M \Phi = I$
- Métodos numéricos contra el exacto
- Entradas inválidas → ValueError

Por qué ahora

Antes de publicar, cada cambio debe dejar las 14 pruebas en verde.

ETAPA 6 · PRUEBAS · 10 MIN

Cómo se lee una prueba

CÓDIGO	QUÉ SIGNIFICA
`def test_…():`	pytest busca solo las funciones cuyo nombre empieza con test_ en la carpeta tests/ y las ejecuta todas.
`assert condición`	«Afirmo que esto es verdad». Si es falso, la prueba falla y pytest muestra qué valores no coincidieron.
`np.allclose(a, b)`	Compara decimales con tolerancia: en un computador 0.1 + 0.2 no es exactamente 0.3.
`pytest.raises(ValueError)`	Comprueba que el programa sí proteste ante datos inválidos: eso también es un comportamiento esperado.

```
tests/test_core.py ......F.......
```

Cada punto es una prueba que pasa; una F es una que falla. Al final pytest resume, por ejemplo, 13 passed, 1 failed.

ETAPA 7 · PYPI · 20 MIN

La estructura que espera PyPI

```
osciladores-acoplados/
├── pyproject.toml
├── README.md
├── LICENSE
├── src/
│   └── osciladores_acoplados/
│       ├── __init__.py
│       └── core.py
├── tests/test_core.py
├── GUI.py            ← no se publica
└── ejemplo_consola.py
```

```
[project]
name = "osciladores-acoplados"
version = "0.1.0"
requires-python = ">=3.9"
dependencies = ["numpy>=1.22"]

[project.optional-dependencies]
gui = ["matplotlib>=3.5"]

[tool.setuptools.packages.find]
where = ["src"]
```

Se instala como **osciladores-acoplados** y se importa como **osciladores_acoplados**. Sólo lo que está en src/ entra al paquete.

ETAPA 7 · PYPI · 20 MIN

Instalar el proyecto como un profesional

EL ERROR QUE TODOS VERÁN

```
PS> python .\GUI.py
Traceback (most recent call last):
  File "GUI.py", line 21, in <module>
ModuleNotFoundError:
  No module named 'osciladores_acoplados'
```

La librería vive en src/ y Python no sabe que existe. No se arregla moviendo archivos ni tocando sys.path: se instala.

LA SOLUCIÓN, UNA VEZ POR PROYECTO

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[gui,dev]"
python GUI.py
```

CÓDIGO	QUÉ SIGNIFICA
`.venv`	Un Python propio del proyecto, aislado de los demás.
`-e`	Enlace a src/: los cambios en core.py se ven al instante.
`[gui,dev]`	Añade matplotlib, pytest, build y twine.

En cada terminal nueva: activar el entorno · el prompt debe mostrar (.venv)

ETAPA 7 · PYPI · 20 MIN

Seis pasos para publicar

1

Nombre y versión

`editar pyproject.toml`

2

Construir

`python -m build`

3

Revisar

`twine check dist/*`

4

Ensayar en TestPyPI

```
twine upload --repository
testpypi dist/*
```

5

Probar la instalación

```
pip install -i
https://test.pypi.org/simple/ …
```

6

Publicar

`twine upload dist/*`

Usuario: __token__ · contraseña: el token de API · cada versión se sube una sola vez

ETAPA 7 · PYPI · 20 MIN

Paso 1: la información va en pyproject.toml

```toml
[project]
name = "osciladores-acoplados-sunombre"   # URL y pip install
version = "0.1.0"                         # cambia en cada envío
description = "Osciladores acoplados 1D"  # subtítulo de la página
readme = "README.md"                      # cuerpo de la página
authors = [{ name = "Su Nombre", email = "su@correo.com" }]
license = "MIT"
dependencies = ["numpy>=1.22"]            # pip las instala solo

[project.urls]
Homepage = "https://github.com/su_usuario/osciladores-acoplados"
```

Mismo archivo, dos destinos

TestPyPI y PyPI leen el mismo pyproject.toml. Sólo cambia el comando twine.

Versión en dos lugares

version en pyproject.toml y __version__ en __init__.py deben decir lo mismo.

Archivo: osciladores-acoplados/pyproject.toml · se edita con cualquier editor de texto

ETAPA 7 · PYPI · 20 MIN

Pasos 2 y 3: construir y revisar en consola

```
# 0. Ubicarse en el proyecto y activar el entorno
cd C:\Users\Usuario\proyectos\osciladores-acoplados
.\.venv\Scripts\Activate.ps1

# 1. Borrar construcciones anteriores
Remove-Item -Recurse -Force dist -ErrorAction SilentlyContinue

# 2. Construir el paquete
python -m build

# 3. Revisar que PyPI lo aceptará
python -m twine check dist/*
```

RESULTADO EN LA CARPETA DIST/

CÓDIGO	QUÉ SIGNIFICA
`….tar.gz`	Código fuente (sdist).
`….whl`	La rueda (wheel): lo que pip instala.
`PASSED`	twine check aprobó los metadatos y el README.

Todos los comandos se escriben en PowerShell, dentro de la carpeta del proyecto, con (.venv) activo

ETAPA 7 · PYPI · 20 MIN

Antes de subir: dos cuentas y dos tokens

	TestPyPI · ensayo	PyPI · real
Sitio	test.pypi.org	pypi.org
Cuenta	Registro propio	Otro registro, independiente
Seguridad	2FA obligatoria	2FA obligatoria
Token	Account settings → API tokens	Account settings → API tokens
Alcance	1.ª vez: Entire account	1.ª vez: Entire account

DÓNDE SE GUARDAN LOS TOKENS

`C:\Users\Usuario\.pypirc`

```
[testpypi]
  username = __token__
  password = pypi-AgENdGVzdC5weXBp...

[pypi]
  username = __token__
  password = pypi-AgEIcHlwaS5vcmc...
```

Cada token va en su sección. Nunca suba este archivo a GitHub.

Los tokens de ejemplo están recortados: los reales son mucho más largos

ETAPA 7 · PYPI · 20 MIN

Pasos 4 y 5: ensayo en TestPyPI

```
# 4. Subir al repositorio de ensayo
python -m twine upload --repository testpypi dist/*
#    → https://test.pypi.org/project/osciladores-acoplados-sunombre/

# 5. Probar en un entorno limpio, en otra carpeta
python -m venv C:\temp\prueba
C:\temp\prueba\Scripts\Activate.ps1
python -m pip install `
    --index-url https://test.pypi.org/simple/ `
    --extra-index-url https://pypi.org/simple/ `
    osciladores-acoplados-sunombre
python -c "import osciladores_acoplados as oa; print(oa.__version__)"
```

--repository testpypi

Toma el token de la sección [testpypi] de .pypirc.

--extra-index-url

NumPy no está en TestPyPI: pip lo descarga de PyPI.

El acento grave `

Continúa el comando en la línea siguiente en PowerShell.

TestPyPI se puede usar todas las veces que haga falta; también exige una versión nueva en cada subida

ETAPA 7 · PYPI · 20 MIN

Paso 6: la versión final en PyPI

```
# Volver al entorno del proyecto
deactivate
cd C:\Users\Usuario\proyectos\osciladores-acoplados
.\.venv\Scripts\Activate.ps1

# 6. Subir los MISMOS archivos de dist/ a PyPI
python -m twine upload dist/*
#    → https://pypi.org/project/osciladores-acoplados-sunombre/

# Desde ahora, cualquier persona puede instalarlo
pip install "osciladores-acoplados-sunombre[gui]"
```

Para la siguiente versión

1. Subir la versión en pyproject.toml y __init__.py
2. Borrar dist/
3. python -m build y twine check
4. Subir a TestPyPI y probar
5. Subir a PyPI

Sin vuelta atrás

PyPI no acepta dos veces la misma versión, ni siquiera después de borrarla.

Sin --repository, twine sube a PyPI · el import sigue siendo osciladores_acoplados

CIERRE · 5 MIN

TAREA:

1. Amortiguamiento

Agregar una fuerza $-\gamma v$. ¿Qué estrategia deja de ser válida y por qué?

2. Resonancia

Forzar una masa con $F_0 \cos(\Omega t)$ y barrer Ω cerca de cada ω_j.

3. Un integrador más

Euler-Cromer o Leapfrog, registrado en la fábrica y con su prueba.

4. Otra interfaz

Una línea de comandos con argparse o un notebook, sin tocar core.py.

Cada reto se entrega como una nueva versión del paquete: 0.2.0, 0.3.0…